



Laporan Praktikum Algoritma & Pemrograman
Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251235
Nama Lengkap	NATALIA GRECIA PUTRI
Minggu ke / Materi	11 / Tipe Data Dictionary

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

Dictionary adalah struktur data dalam Python yang memiliki kesamaan dengan list, namun jauh lebih general dalam penggunaannya. Jika pada list setiap elemen diakses melalui indeks berupa bilangan bulat, maka pada *dictionary* indeks yang digunakan dapat berupa tipe data apapun, termasuk string dan angka. Setiap anggota dictionary terdiri dari sepasang kunci dan nilai (key-value pair), di mana setiap kunci harus bersifat unik, tidak diperbolehkan ada dua kunci yang sama dalam satu dictionary. Dengan demikian, dictionary dapat dipahami sebagai sebuah pemetaan antara kumpulan kunci dan kumpulan nilai yang saling berkorespondensi. Hubungan antara kunci dan nilai ini disebut pula sebagai item. Sebagai gambaran, dictionary bisa digunakan untuk membangun kamus terjemahan, misalnya dari bahasa Inggris ke bahasa Spanyol, di mana setiap kata dalam bahasa Inggris dipetakan ke satu terjemahannya dalam bahasa Spanyol.

Fungsi `dict()` digunakan untuk menginisialisasikan dictionary kosong. Karena `dict` adalah fungsi bawaan (*built-in*) Python, hindari penggunaannya sebagai nama variabel.

```
>>> eng2sp = dict()
>>> print(eng2sp)
{}

```

Kurung kurawal `{}` melambangkan dictionary kosong. Untuk menambahkan item baru, menggunakan kurung siku `[]`.

```
>>> eng2sp['one'] = 'uno'

```

Kode diatas membuat item baru yang memetakan kunci 'one' ke nilai 'uno'. Jika dictionary tersebut dicetak, pasangan kunci-nilai ini akan terlihat.

```
>>> print(eng2sp)
{'one': 'uno'}

```

Format pencetakan ini juga berlaku sebagai format input. Misalnya, saat membuat dictionary dengan tiga item sekaligus.

```
>>> eng2sp = {'one': 'unc', 'two': 'dos', 'three': 'tres'}
>>> print(eng2sp)
{'one': 'unc', 'three': 'tres', 'two': 'dos'}
```

Perhatikan bahwa urutan pasangan kunci-nilai pada output tidak sama dengan urutan saat input. Urutan item dalam dictionary bersifat acak atau tidak bisa diprediksi. Hal ini bukanlah masalah karena elemen dictionary tidak diakses menggunakan indeks integer, melainkan dengan mencari nilai berdasarkan kuncinya.

```
>>> print(eng2sp['two'])
'dos'
```

Kunci 'two' akan selalu berasosiasi dengan nilai 'dos' tanpa peduli urutannya. Namun, jika dicoba mengakses kunci yang tidak terdaftar, sistem akan memunculkan pesan error:

```
>>> print(eng2sp['four'])
KeyError: 'four'
```

Fungsi len() pada dictionary akan menghitung jumlah pasangan kunci-nilai yang ada:

```
>>> len(eng2sp)
3
```

Operator in pada dictionary akan mengembalikan nilai boolean True atau False berdasarkan keberadaan kunci (bukan nilai) di dalam dictionary tersebut.

```
>>> 'one' in eng2sp
True
>>> 'unc' in eng2sp
False
```

Untuk memeriksa keberadaan suatu nilai, dapat memanfaatkan method values(). Method ini mengembalikan kumpulan nilai dapat dikonversi menjadi list agar bisa digunakan bersama operator in.

```
>>> vals = list(eng2sp.values())
>>> 'unc' in vals
True
```

Cara kerja operator `in` berbeda antara list dan dictionary. Pada list, algoritma pencarian bersifat linear, sehingga waktu pencarian akan memanjang seiring bertambahnya ukuran list. Sebaliknya, dictionary menggunakan algoritma *Hash Table* yang sangat efisien; operator `in` membutuhkan waktu konstan yang sama cepatnya tidak peduli seberapa besar ukuran dictionary.

Dictionary sebagai set penghitung (*counters*)

kita diberi tugas untuk menghitung frekuensi kemunculan huruf dalam sebuah string. Ada beberapa pendekatan yang bisa dilakukan:

1. Membuat 26 variabel untuk setiap huruf alfabet, lalu menggunakan kondisional berantai untuk menambah hitungan.
2. Membuat list berisi 26 elemen, mengkonversi karakter menjadi angka menggunakan fungsi bawaan, lalu menggunakan angka tersebut sebagai indeks untuk menambah hitungan.
3. Membuat dictionary dengan karakter sebagai kunci dan jumlah hitungan sebagai nilai.

Ketiga opsi tersebut menghasilkan perhitungan yang sama, tetapi dengan cara komputasi yang berbeda. Model dictionary dinilai paling praktis karena kita tidak perlu mendefinisikan huruf apa yang akan muncul terlebih dahulu; dictionary akan secara dinamis menyediakan ruang untuk huruf baru saat huruf tersebut muncul.

Berikut implementasinya:

```
1 word = 'brontosaurus'
2 d = dict()
3 for c in word:
4     if c not in d:
5         d[c] = 1
6     else:
7         d[c] = d[c] + 1
8 print(d)
```

Model komputasi semacam ini disebut sebagai *histogram*, yang merupakan istilah statistik untuk set penghitung frekuensi. Perulangan `for` akan menelusuri setiap karakter string. Jika karakter `c` belum ada di dictionary, sistem akan membuat item baru dengan kunci `c` dan nilai awal 1. Jika sudah ada, nilai dari kunci `c` akan ditambahkan 1.

Output program: `{'b': 1, 'r': 2, 'o': 2, 'n': 1, 't': 1, 's': 2, 'a': 1, 'u': 2}`

Histogram tersebut menunjukkan bahwa huruf "a" dan "b" muncul 1 kali, "o" muncul 2 kali, dan seterusnya.

Python menyediakan method `get()` yang mempermudah penulisan histogram. Method ini mengambil dua parameter: kunci dan nilai *default*. Jika kunci ada di dictionary, ia mengembalikan nilai kunci tersebut; jika tidak, ia mengembalikan nilai *default*.

```
>>> counts = { 'chuck' : 1 , 'annie' : 42, 'jan': 100}
>>> print(counts.get('jan', 0))
100
>>> print(counts.get('tim', 0))
0
```

Penggunaan `get()` untuk menyederhanakan perhitungan ini telah menjadi pola umum yang sangat lazim digunakan dalam pemrograman Python. Meski lebih ringkas, fungsinya sama persis dengan loop yang menggunakan `if` dan operator `in`.

Dictionary dan File

Dictionary juga sangat berguna untuk menghitung frekuensi kemunculan kata-kata dalam sebuah file teks. Programnya bekerja dengan membaca file baris demi baris, lalu memecah setiap baris menjadi kata-kata individual. Proses pencacahan dilakukan menggunakan dua perulangan bersarang: perulangan untuk iterasi tiap baris dalam file, dan perulangan dalam untuk iterasi tiap kata dalam baris. Setiap kata yang ditentukan akan dicatat dalam dictionary beserta jumlah kemunculan.

```

1  fname = input('Enter the file name: ')
2  try:
3      fhand = open(fname)
4  except:
5      print('File cannot be opened:', fname)
6      exit()
7
8  counts = dict()
9  for line in fhand:
10     words = line.split()
11     for word in words:
12         if word not in counts:
13             counts[word] = 1
14         else:
15             counts[word] += 1
16
17  print(counts)

```

Dalam implementasinya, penulisan `counts[word] += 1` merupakan bentuk singkat dari `counts[word] = counts[word] + 1`. Pola singkat serupa juga tersedia untuk operasi pengurangan (`-=`), perkalian (`*=`), dan pembagian (`/=`). Penggunaan notasi ini tidak hanya membuat kode lebih ringkas, tetapi juga lebih mudah dibaca.

Looping dan Dictionary

Ketika sebuah perulangan `for` diterapkan pada dictionary, yang ditelusuri secara default adalah kunci-kuncinya, bukan nilai-nilainya.

```

1  counts = { 'chuck' : 1 , 'annie' : 42, 'jan': 100}
2  for key in counts:
3      print(key, counts[key])

```

Untuk mengakses nilai yang berkorespondensi dengan setiap kunci, diperlukan operator indeks `[ ]` di dalam badan perulangan. Dengan cara ini, setiap iterasi dapat mencetak pasangan kunci beserta nilainya.

Apabila diperlukan penelusuran kunci secara alfabetikal, langkahnya adalah mengambil seluruh kunci menggunakan method `.keys()`, mengonversinya ke list, lalu mengurutkannya dengan `.sort()`. Setelah itu, perulangan dilakukan pada list yang sudah terurut tersebut, dan nilai dari masing-masing kunci diakses langsung dari dictionary. Hasilnya adalah tampilan pasangan kunci-nilai dalam urutan alfabet yang rapi.

```

1 counts = { 'chuck' : 1 , 'annie' : 42, 'jan': 100}
2 for key in counts:
3     if counts[key] > 10 :
4         print(key, counts[key])

```

Advanced Text Parsing

Dalam pemrosesan teks yang lebih lanjut, fungsi `split()` memisahkan kalimat menjadi kata-kata berdasarkan spasi. Namun, pendekatan dasar ini memiliki keterbatasan: kata yang diikuti tanda baca, seperti "soft!" akan dianggap berbeda dari "soft", begitu pula "Who" dan "who" yang dibedakan berdasarkan huruf kapitalnya. Akibatnya, kata-kata yang sejatinya sama akan tercatat sebagai entri yang berbeda dalam dictionary.

Untuk mengatasi masalah ini, Python menyediakan beberapa solusi. Method `.lower()` digunakan untuk menyeragamkan semua huruf menjadi huruf kecil sehingga perbedaan kapitalisasi tidak lagi mempengaruhi penghitungan. Sementara itu, untuk menghapus tanda baca, dapat dimanfaatkan modul `string` yang menyediakan konstanta `string.punctuation` berisi semua karakter tanda baca. Penghapusan tanda baca dilakukan menggunakan method `.translate()` yang dikombinasikan dengan `str.maketrans()`, dan dianggap sebagai cara yang paling bersih dan efisien untuk membersihkan teks sebelum diproses lebih lanjut.

```

1 import string
2
3 fname = input('Enter the file name: ')
4 try:
5     fhand = open(fname)
6 except:
7     print('File cannot be opened:', fname)
8     exit()
9
10 counts = dict()
11 for line in fhand:
12     line = line.rstrip()
13     line = line.translate(line.maketrans('', '', string.punctuation))
14     line = line.lower()
15     words = line.split()
16     for word in words:
17         if word not in counts:
18             counts[word] = 1
19         else:
20             counts[word] += 1
21
22 print(counts)

```

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

SOAL 1

https://github.com/grec12/71251235_nataliagrecia.git

```
1 dictionary = {1: 10, 2: 20, 3: 30, 4: 40, 5: 50, 6: 60}
2
3 print(f"{'key':<5} {'value':<7} {'item':<5}")
4
5 for key, value in dictionary.items():
6     print(f"{'key':<5} {'value':<7} {'key':<5}")
7
```

key	value	item
1	10	1
2	20	2
3	30	3
4	40	4
5	50	5
6	60	6

Penjelasan:

- `dictionary = {...}` : Inisialisasi dictionary yang berisi beberapa pasangan kunci dan nilai.
- `print(f"...")` : Menampilkan judul tabel dengan pengaturan format rata kiri serta lebar kolom yang sudah ditentukan.
- `print("-----")` : Mencetak garis horizontal sebagai pembatas tabel.
- `for key, value in dictionary.items():` : Melakukan iterasi untuk mengambil setiap pasangan kunci-nilai yang ada di dalam dictionary.
- `print(f"...")` : Menampilkan data ke dalam bentuk tabel yang terdiri dari kolom kunci, nilai, serta pengulangan kunci.
- Pada kolom item, yang ditampilkan hanya kunci saja, bukan keseluruhan pasangan (kunci, nilai).

SOAL 2

```
1  Lista = ['red', 'green', 'blue']
2  Listb = [red '#FF0000', green '#008000', blue '#0000FF']
3
4  hasil = {}
5
6  for i in range(len(Lista)):
7      |   hasil[Lista[i]] = Listb[i]
8
9  print(hasil)
```

```
{'red': '#FF0000', 'green': '#008000', 'blue': '#0000FF'}
```

Penjelasan:

- Lista dan Listb : Dua buah list yang masing-masing menyimpan daftar nama warna dan kode heksadesimal warna tersebut.
- hasil = {} : Membuat sebuah dictionary yang masih kosong sebagai wadah hasil.
- for i in range(len(Lista)): : Perulangan yang berjalan sebanyak jumlah elemen pada Lista, menggunakan indeks sebagai acuan.
- hasil[Lista[i]] = Listb[i] : Memasukkan pasangan baru ke dalam dictionary, di mana nama warna dari Lista berperan sebagai kunci, dan kode warna dari Listb berperan sebagai nilai.
- print(hasil) : Menampilkan dictionary yang sudah terbentuk dari hasil pemetaan warna ke kode warnanya.

SOAL 3

```
1 filename = input("Masukkan nama file: ")
2 email_counts = {}
3
4 with open(filename, 'r') as file:
5     for line in file:
6         if line.startswith('From'):
7             parts = line.split()
8             email = parts[1]
9             if email in email_counts:
10                 email_counts[email] += 1
11             else:
12                 email_counts[email] = 1
13
14 print(email_counts)
```

```
Masukkan nama file: mbox-short.txt
{'stephen.marquard@uct.ac.za': 4, 'louis@media.berkeley.edu': 6, 'zqian@umich.edu': 8, 'rjlowe@iupui.edu': 4, 'cwen@iupui.edu': 10,
'gsilver@umich.edu': 6, 'wagnermr@iupui.edu': 2, 'antranig@caret.cam.ac.uk': 2, 'gopal.ramasammycook@gmail.com': 2, 'david.horwitz@uct.ac.za': 8, 'ray@media.berkeley.edu': 2}
```

Penjelasan:

- `filename = input(...)` : Meminta pengguna untuk memasukkan nama file yang akan dibaca.
- `email_counts = {}` : Menyiapkan dictionary kosong yang akan digunakan untuk menyimpan jumlah kemunculan setiap email.
- `with open(...)` : Membuka file dalam mode baca secara aman.
- `for line in file:` : Membaca isi file secara bertahap, baris demi baris.
- `if line.startswith('From ')` : Menyaring baris sehingga hanya baris yang dimulai dengan kata 'From ' (dengan spasi) yang diproses, mengabaikan baris seperti 'From:'.
- `parts = line.split()` : Memecah satu baris teks menjadi beberapa bagian kata dan menyimpannya dalam bentuk list.
- `email = parts[1]` : Mengambil alamat email yang terletak pada posisi indeks ke-1.
- `if email in email_counts:` : Memeriksa keberadaan email tersebut dalam dictionary:
 - Jika sudah tercatat : nilai hitungannya ditambah satu.
 - Jika belum ada : dicatat sebagai entri baru dengan nilai awal 1.
- `print(email_counts)` : Mencetak dictionary yang berisi data frekuensi kemunculan masing-masing email.

SOAL 4

```
1 filename = input("Masukkan nama file: ")
2 domain_counts = {}
3
4 with open(filename, 'r') as file:
5     for line in file:
6         if line.startswith('From'):
7             parts = line.split()
8             email = parts[1]
9             domain = email.split('@')[1]
10            if domain in domain_counts:
11                domain_counts [domain] += 1
12            else:
13                domain_counts [domain] = 1
14
15 print(domain_counts)
```

```
Masukkan nama file: mbox-short.txt
{'uct.ac.za': 12, 'media.berkeley.edu': 8, 'umich.edu': 14, 'iupui.edu': 16, 'caret.cam.ac.uk': 2, 'gmail.com': 2}
```

Penjelasan:

- `filename = input(...)` : Meminta pengguna untuk mengetikkan nama file yang ingin diproses.
- `domain_counts = {}` : Membuat dictionary kosong untuk menghitung berapa kali setiap domain email muncul.
- `with open(...)` : Membuka file teks dalam mode baca.
- `for line in file:` : Menelusuri isi file per baris.
- `if line.startswith('From ')` : Memfilter agar hanya baris yang diawali dengan 'From ' yang diproses lebih lanjut.
- `parts = line.split()` : Memisahkan baris menjadi kumpulan kata berdasarkan spasi.
- `email = parts[1]` : Mengambil alamat email dari indeks kedua pada list hasil pemisahan.
- `domain = email.split('@')[1]` : Memecah email pada tanda '@' dan mengambil bagian setelahnya, yaitu nama domain.
- Pengecekan keberadaan domain dalam `domain_counts`:
 - Apabila domain sudah tersedia : nilainya ditambah satu.
 - Apabila domain belum ada : ditambahkan sebagai entri baru dengan nilai 1.
- `print(domain_counts)` : Menampilkan hasil perhitungan frekuensi dari setiap domain email yang ditemukan.